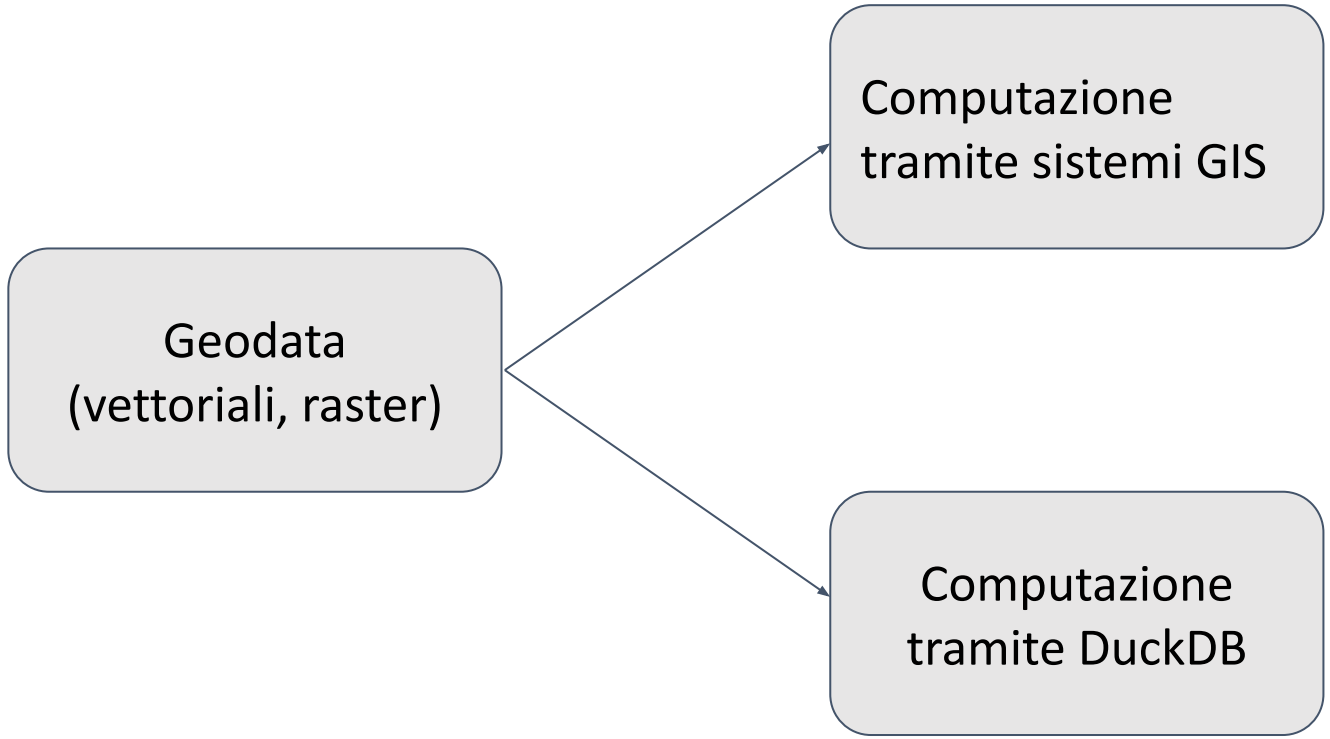
UNIVERSITÀ DI TORINO

Generazione di tiles vettoriali e visualizzazione di dati geospaziali tramite DuckDB e Maplibre

Introduzione: Duckdb tiling

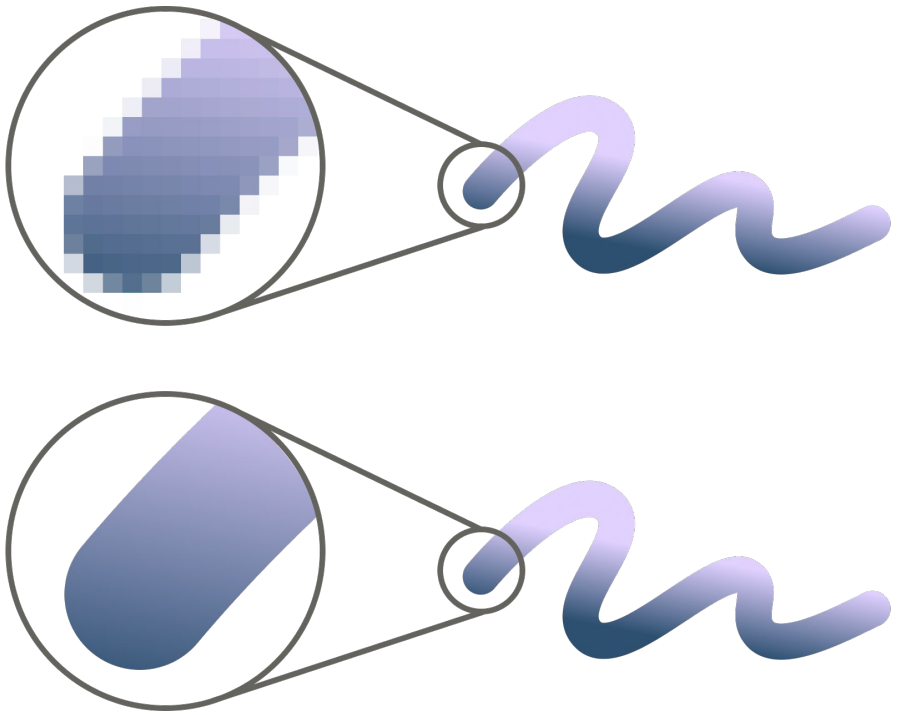
Quando parliamo di **geodata**, ci riferiamo ad una macro-tipologia di dati *geospaziali* grezzi che, computati in un determinato modo, vengono visualizzati come immagini su una mappa.



Geodati: Raster vs Vector

UNIVERSITÀ DI TORINO

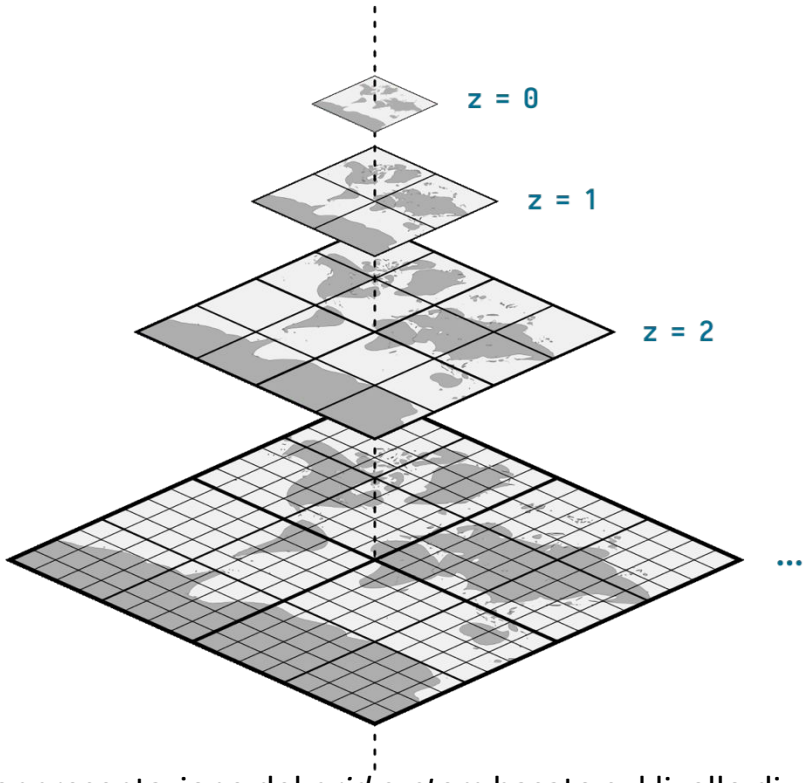
I geodata si suddividono in due tipologie: **Raster** e **Vector**.
I **Raster** sono immagini vere e proprie, in formati come *jpeg*, *png*…
I **Vector**, invece, sono dati che "costruiscono" l'immagine, definiti grazie a primitive geometriche come *punti*, *linee* e *poligoni*.



La stessa immagine, ma in formato Raster, in alto, ed in formato vector, in basso. All'aumentare della risoluzione, i Raster "sgranano", cosa che non succede per i Vector.

Tiles e formato: PBF ed MVT

Il progetto si sofferma sui dati *vettoriali*. I dati vettoriali, per essere visualizzati su mappa, seguono il *grid system* di tiles vettoriali. Ogni geometria viene geolocalizzata in un quadrato di mappa, detto **tile**. Le *tiles* sono veri e propri **file binari**, in formato **MVT** (*Mapbox Vector Tile*) che segue lo standard per i formati vettoriali di Google **PBF** (*ProtocolBuffer Binary Format*). I due formati di dati vettoriali che utilizziamo sono **GeoJSON** (testuale) e **Parquet** (binario).

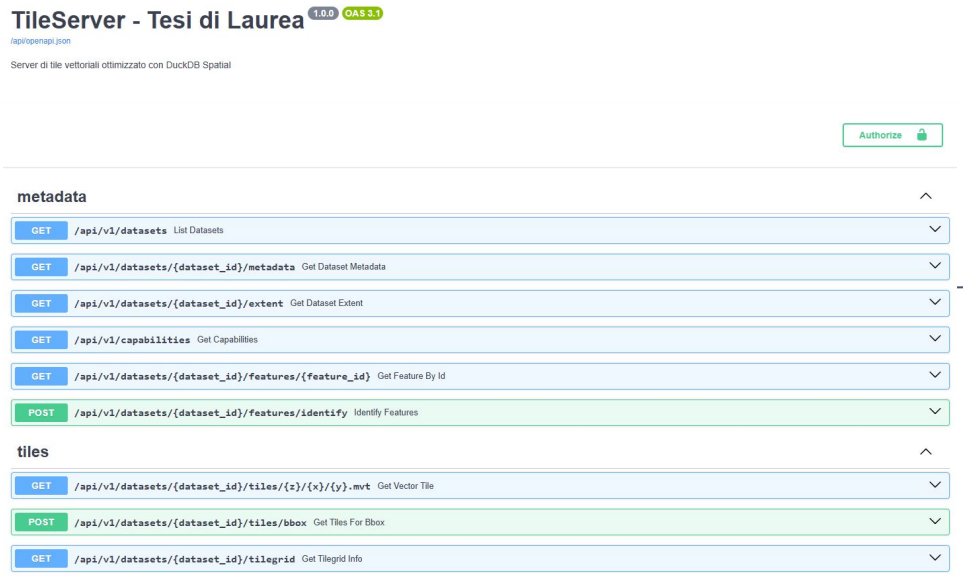


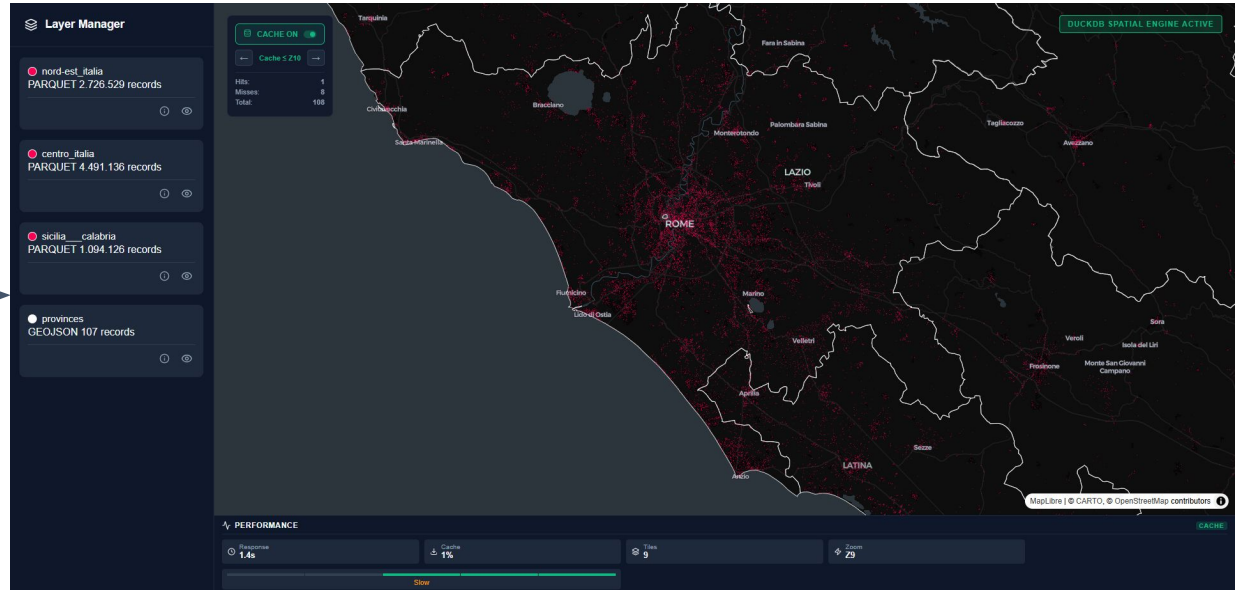
Rappresentazione del *grid system* basato sul livello di zoom per la suddivisione della mappa in tile. Si parte da una sola tile rappresentabile per lo zoom più basso, aumentando a 4^z tiles per ogni livello (1, 4, 16…)

Obiettivo del progetto

UNIVERSITÀ DI TORINO

L'obiettivo del progetto è quello di costruire un Tileserver dinamico che restituisca tiles vettoriali ad una mappa del mondo che le visualizzerà con precisione ed efficienza, testando a pieno le potenzialità di DuckDB





Il risultato del progetto: a sinistra, la documentazione delle API del Tileserver, per poter testare ed utilizzare i servizi che il server offre. A destra la mappa, che visualizza le vector tiles caricate grazie alle API messe a disposizione dal server (i dati sono rappresentati dai punti rossi e linee bianche).

Stack tecnologico

Questo progetto *full-stack* è composto di tre livelli: la mappa **MapLibre**, montata sul *framework* per il frontend **SvelteKit**, effettua le richieste alle API **FastAPI**, che a loro volta effettuano le query a **DuckDB** per ottenere le geometrie e standardizzarle per costruire il file *MVT* da restituire alla mappa frontend.

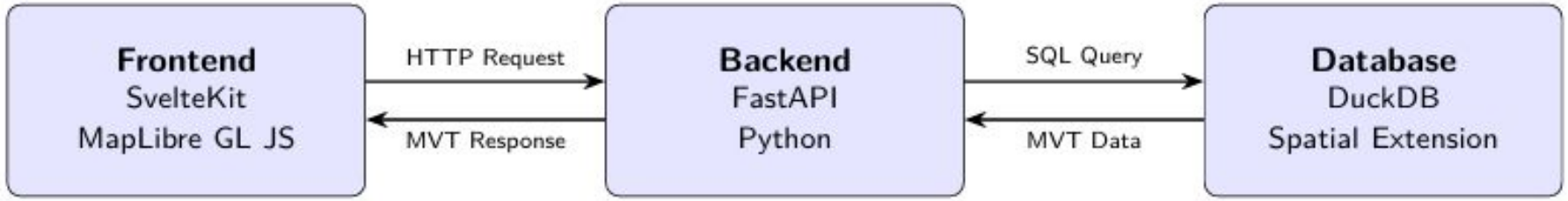
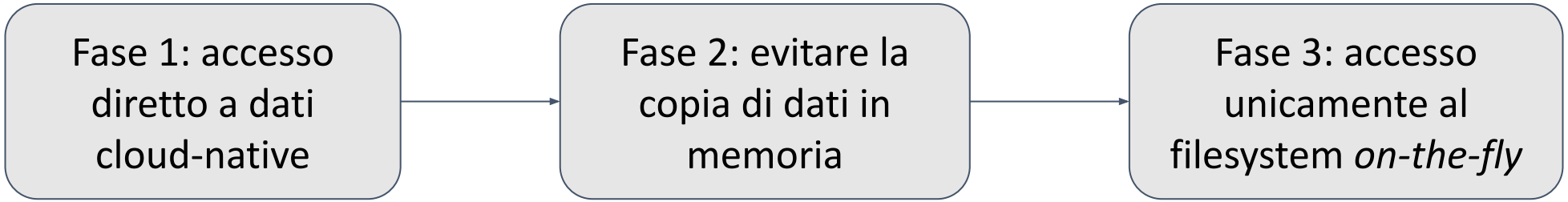


Diagramma che rappresenta lo scheletro architetturale del progetto, con i vari scambi di dati

Le direttive: il paradigma zero-copy

UNIVERSITÀ DI TORINO

Per un progetto *full-stack* dove trattiamo geodati, è fondamentale che la *pipeline* segua i principi del paradigma **zero-copy**, che è composto da queste 3 fasi: utilizzo di formati **cloud-native** (come Parquet), **evitare copie** ridondanti di dati in memoria (utilizzando ad esempio *GeoArrow*) ed **accesso** unicamente al **filesystem** per la **fruizione delle tiles** *in-real-time*.



Rappresentazione delle 3 fasi del paradigma zero-copy per i geodati

Generazione dinamica di tiles

UNIVERSITÀ DI TORINO

La generazione di tiles *on-the-fly* avviene grazie ad una query effettuata dal backend *python* alla ricezione di una richiesta per l'API principale del server (**/tiles/dataset_id/z/x/y.mvt**). La query restituisce i dati binari corrispondenti alla tile *zxy* richiesta con tutte le geometrie che intersecano quell'area di mappa.

Le tre componenti principali della query mvt

Caching system

UNIVERSITÀ DI TORINO

Il progetto implementa 3 livelli di **cache** a sostegno del meccanismo di generazione *on-the-fly*:

- Cache **browser** (dipendente dal browser utilizzato)
- Cache in **memoria** (*RAM*, cache *LRU*)
- Cache su **disco** (gerarchia di cartelle che contengono le tiles)

```
data/tile_cache/
 dataset_1/
      zoom_10/
           500_300.mvt
           501_300.mvt
      zoom_11/
 dataset_2/
```

Un esempio di cache su disco è questa in figura. Le cartelle sono organizzate per dataset e zoom, e contengono le tiles pre-generate: è possibile pre-generare le tiles grazie ad una specifica API del backend.

Strategia di integrazione

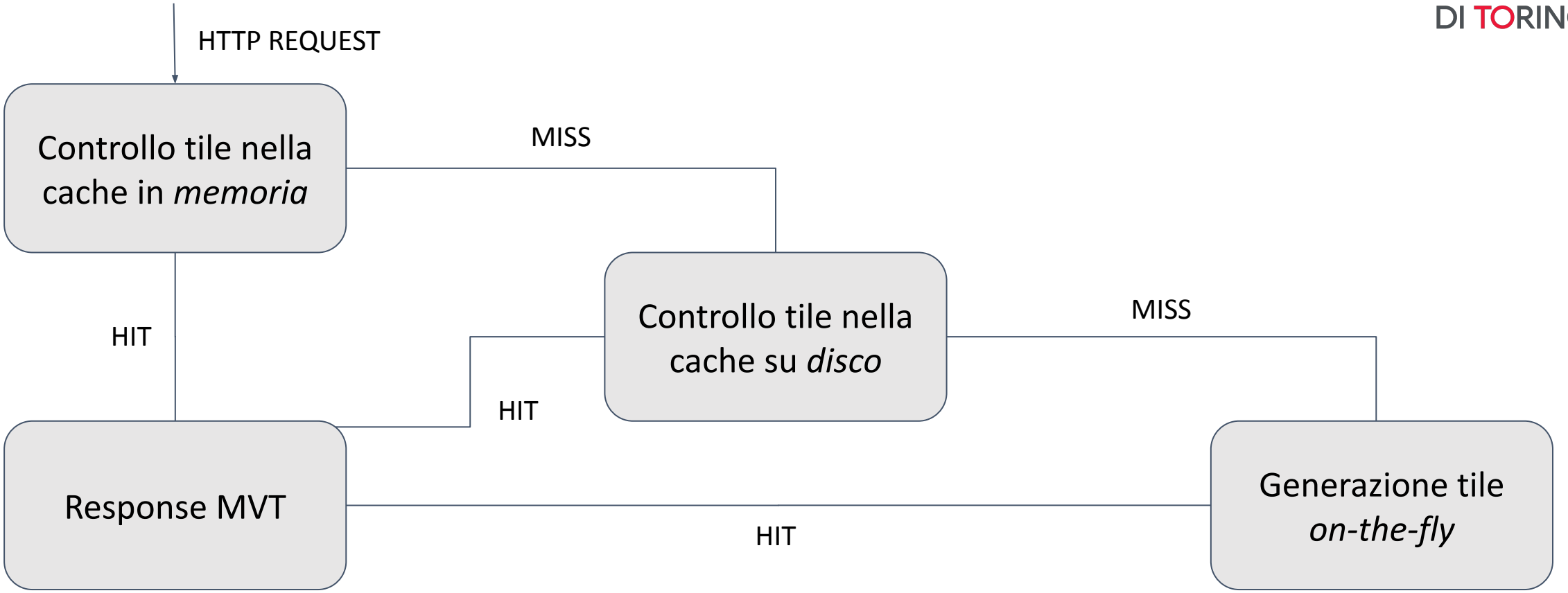


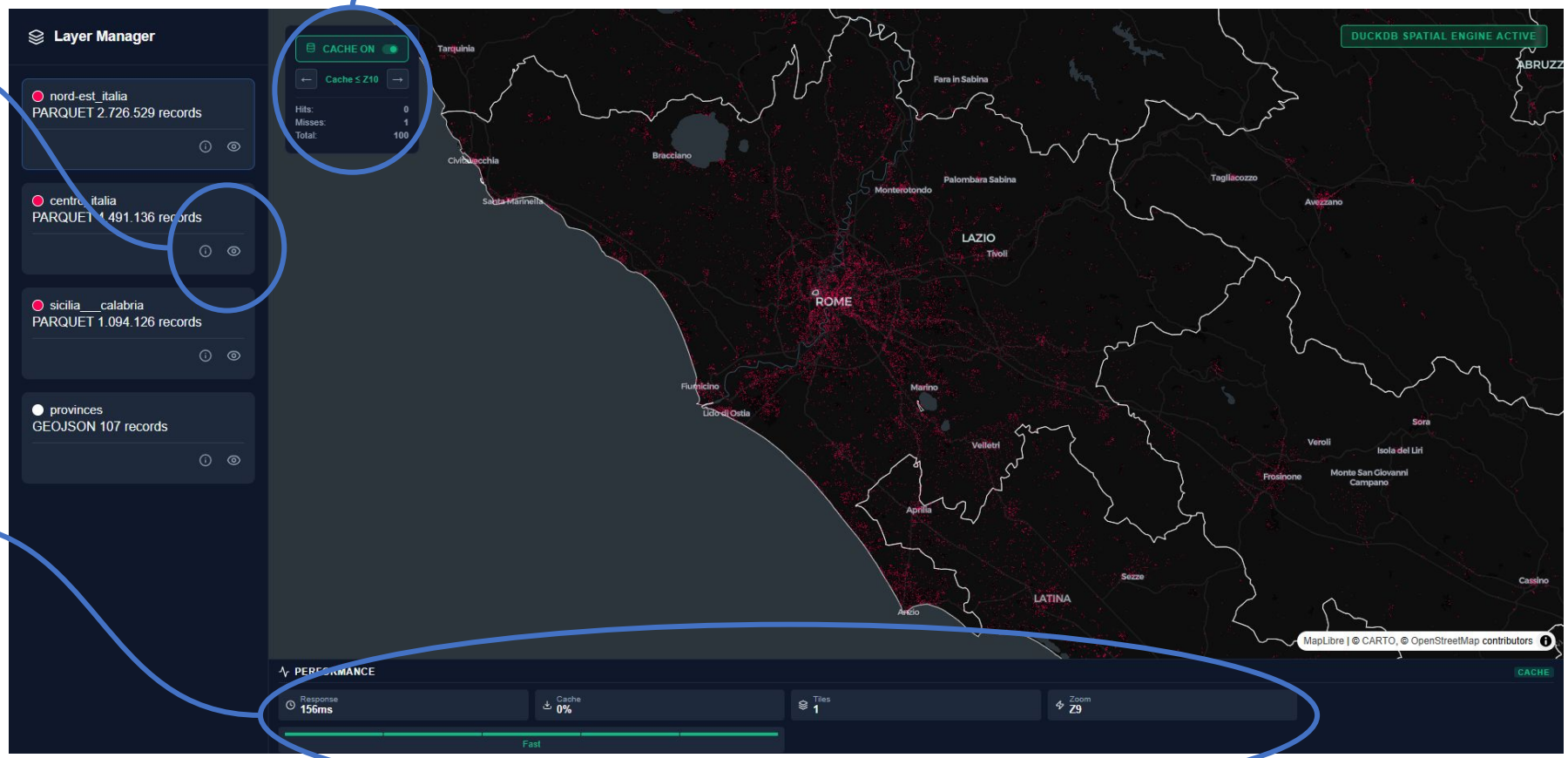
Diagramma per la scelta della strategia da adottare per restituire la tile vettoriale richiesta

Features aggiuntive

Pannello per la gestione dell'utilizzo della cache

Mostra/Escludi il dataset dalla visualizzazione

Pannello per mostrare i dettagli e le performance, come il tempo medio di response, la percentuale di hit, il numero di tiles e lo zoom attuale



Performance del sistema

UNIVERSITÀ DI TORINO

Tipo di tile	Dataset piccolo (5MB)	Dataset medio (50MB)	Dataset grande (500MB)
Cache hit (disco)	<15ms	<25ms	<60ms
Cache hit (memoria)	<1ms	<1ms	<1ms
Generazione (zoom 10)	250-450ms	500-1000ms	1200-2000ms
Generazione (zoom 14)	100-250ms	250-500ms	500-1200ms
Precomputazione (zoom 0-5)	10-20s	30-60s	60-90s

* I tempi di risposta sono da mettere in relazione non solo con la dimensione del dataset, ma anche con la sua distribuzione spaziale. Dataset con geometrie molto concentrate in poche tile possono generare tempi più elevati a zoom bassi (come nel caso dei nostri file parquet, rappresentanti le strutture italiane), mentre dataset più uniformemente distribuiti tendono a mantenere tempi più stabili.

Criticità, limitazioni e conclusioni

UNIVERSITÀ DI TORINO

La maggiore **criticità** del progetto riguarda il mancato utilizzo di un formato di archiviazione come **GeoArrow**, che rende parzialmente incompleto il punto 2 del paradigma *zero-copy.*

Come **limitazione** principale notiamo **l'overhead computazionale** dato dall'utilizzo di funzioni spaziali come *ST_Intersect*, *ST_AsMVTGeom* …

L'esplorazione di questa nuova frontiera tecnologica, tuttavia, è avvenuta con **successo**: la conclusione a cui siamo arrivati è di certo la **validità** di un motore di calcolo analitico come **DuckDB** anche nel campo della generazione di tiles vettoriali *on-the-fly*, rendendolo un'alternativa con grande potenziale per quanto riguarda il *tiling* vettoriale.

UNIVERSITÀ
DI TORINO

UNIVERSITÀ
DI TORINO

Grazie